

Scribed by:

1. Kanishka Singh (2020MT60880)
2. Yash Sajjansingh Chavan (2021MT60966)
3. Prabhat Babu (2021MT10255)
4. Garv Gupta (2021MT60953)
5. Rahul Rajoria (2022MT11947)

1 Recap of Last Lecture

We discussed the limitations of computation and reviewed several **problems that are decidable by a Turing Machine**, such as:

- A_{DFA} : Acceptance problem for DFA
- E_{DFA} : Emptiness problem for DFA
- EQ_{DFA} : Equivalence problem for DFA
- A_{NFA} : Acceptance problem for NFA
- A_{REG} : Acceptance problem for regular grammar
- A_{CFG} : Acceptance problem for CFG
- E_{CFG} : Emptiness problem for CFG

We further noted that the **equivalence problem for CFG**, EQ_{CFG} , was **undecidable**.

2 Overview of This Lecture

2.1 A language that is not Turing decidable

We will introduce the **acceptance problem for Turing Machines**, denoted as A_{TM} . We will first show that A_{TM} is **Turing-recognizable**. Following this, we will present a proof by contradiction to demonstrate that A_{TM} is **undecidable**.

2.2 Languages that are not even Turing recognizable

Finally, we will prove that **not all languages are Turing-recognizable**. We provide a **concrete example** of such a language which is not Turing recognizable using a result stating the **necessary and sufficient condition for a language to be decidable**.

3 A Turing Undecidable Language

3.1 The Acceptance Problem for Turing Machines (A_{TM})

As learned in the previous lecture, the acceptance problems for DFA, NFA, regular grammar and CFG turned out to be decidable. The natural question arises that **whether the acceptance problem for Turing machine is decidable**.

We begin by **formally** defining the acceptance problem for Turing Machine.

Definition 1. *The language A_{TM} is the set of pairs $\langle M, w \rangle$ where M is a Turing machine and w is a string that M accepts.*

$$A_{TM} = \{\langle M, w \rangle \mid M \text{ is a TM and } w \in L(M)\}$$

The question is **whether there exists a Turing Machine that can decide** this language. That is, can we construct a TM that, for any given TM M and string w , will always halt and correctly determine if M accepts w ?

It will turn out that A_{TM} **is undecidable**. But, we observe that it is **at least Turing recognizable**.

Theorem 2. *A_{TM} is Turing-recognizable.*

Proof. We can construct a **Turing Machine** U , that recognizes A_{TM} . On **input** $\langle M, w \rangle$, U works as follows:

1. *Simulate M on input w .*
2. *If M enters its accept state, then U accepts.*
3. *If M enters its reject state, then U rejects.*

This machine U **correctly accepts if M accepts w** . However, **if M loops indefinitely on w , U will also loop**. Because it is not guaranteed to halt on all inputs, U is a recognizer, not a decider. $\square$

Theorem 3. *A_{TM} is not decidable.*

Proof. **Assume for the sake of contradiction that A_{TM} is decidable.** This implies there exists a TM H that is a decider for A_{TM} .

For any input $\langle M, w \rangle$, H **always halts** and outputs as follows:

$$H(\langle M, w \rangle) = \begin{cases} \text{Accept} & \text{if } M \text{ accepts } w \\ \text{Reject} & \text{if } M \text{ does not accept } w \end{cases}$$

We **construct a new TM** D , which takes as input the encoding of a single TM, $\langle M \rangle$. It **calls H as a subroutine** and gives it $\langle M, \langle M \rangle \rangle$ as its input.

For any input $\langle M \rangle$, D outputs as follows:

$$D(\langle M \rangle) = \begin{cases} \text{Accept} & \text{if } H(\langle M, \langle M \rangle \rangle) \text{ rejects } \langle M, \langle M \rangle \rangle \\ \text{Reject} & \text{if } H(\langle M, \langle M \rangle \rangle) \text{ accepts } \langle M, \langle M \rangle \rangle \end{cases}$$

Using the definition of H this **reduces to**

$$D(\langle M \rangle) = \begin{cases} \text{Accept} & \text{if } M \text{ does not accept } \langle M \rangle \\ \text{Reject} & \text{if } M \text{ accepts } \langle M \rangle \end{cases}$$

The Contradiction: Consider what happens when we **feed the encoding of D to itself**. We run D with input $\langle D \rangle$.

$$D(\langle D \rangle) = \begin{cases} \text{Accept} & \text{if } D \text{ does not accept } \langle D \rangle \\ \text{Reject} & \text{if } D \text{ accepts } \langle D \rangle \end{cases}$$

This implies that D accepts $\langle D \rangle$ if and only if D rejects $\langle D \rangle$.

Since our initial assumption leads to a contradiction, the **assumption must be false**. Therefore, A_{TM} is undecidable. $\square$

So, we get the **first Turing undecidable language** among the ones seen so far. But, we have not yet seen a language that is not Turing recognizable. It is natural to question whether **are all languages Turing recognizable?** As we show ahead, there exist languages which are not Turing recognizable.

4 Non Turing-Recognizable Languages

We show that there exist languages that cannot even be recognized by a Turing Machine. But before that we need a **helping tool** which we state in the form of a lemma below.

Lemma 1. *The set of infinite binary sequences is uncountable.*

Proof. Let B be the set of all infinite binary sequences. We can show that B is uncountable by using **Cantor's diagonalization argument**.

Assume for the sake of contradiction that **the set B of all infinite binary sequences is countable**. This means we can enumerate all such sequences in a list: $S_1, S_2, S_3, \dots$. Let's represent this list, where s_{ij} is the j -th bit of the i -th sequence S_i :

	1 st bit	2 nd bit	3 rd bit	4 th bit	...
S_1	s_{11}	s_{12}	s_{13}	s_{14}	...
S_2	s_{21}	s_{22}	s_{23}	s_{24}	...
S_3	s_{31}	s_{32}	s_{33}	s_{34}	...
S_4	s_{41}	s_{42}	s_{43}	s_{44}	...
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\ddots$

Now, we **construct a new infinite binary sequence**, S' , which is not in this list. We define the i -th bit of S' , denoted s'_i , by taking the i -th bit of the i -th sequence S_i and **flipping** it. That is:

$$s'_i = 1 - s_{ii}$$

For example, if the diagonal bits $(s_{11}, s_{22}, s_{33}, \dots)$ were $(0, 1, 1, \dots)$, our new sequence S' would begin with $(1, 0, 0, \dots)$.

By its construction, the sequence S' is guaranteed to be **different from every sequence in our list**. For any sequence S_i in the list, S' differs from S_i in at least the i -th position (since $s'_i \neq s_{ii}$).

This is a **contradiction**. We assumed that our list $S_1, S_2, S_3, \dots$ contained *all* infinite binary sequences, yet we have constructed a new infinite binary sequence S' that is not on the list. Therefore, our initial assumption must be false. The set B cannot be put into a list and is therefore uncountable. $\square$

Theorem 4. *There exists a language L that is not Turing recognizable.*

Proof. To show that the set of all Turing machines is countable, we first observe that **the set of all strings Σ^* is countable for any finite alphabet Σ** . With only a finite number of strings of each length, we may form an ordered list of Σ^* by writing down all strings of length 0, then all strings of length 1, then length 2, and so on.

The **set of all Turing machines is countable** because each Turing machine M has an encoding into a string $\langle M \rangle$. If we simply omit those strings that are not legal encodings of Turing machines from our ordered list of all strings, we obtain a list of all Turing machines.

We know that the set of infinite binary sequences B is uncountable. Let $\mathcal{L}$ be **the set of all languages** over the alphabet Σ . We show that $\mathcal{L}$ is **uncountable** by demonstrating a one-to-one correspondence with B .

Let $\Sigma^* = \{s_1, s_2, s_3, \dots\}$ be an enumeration of all strings in Σ^* . Each language $A \subseteq \Sigma^*$ has a unique corresponding sequence in B , called its **characteristic sequence**, χ_A . The i -th bit of this sequence is 1 if $s_i \in A$ and 0 if $s_i \notin A$.

For **example**, if A were the language of all strings starting with a 0 over the alphabet $\{0, 1\}$, its characteristic sequence χ_A would be:

$$\begin{aligned} \Sigma^* &= \{\epsilon, 0, 1, 00, 01, 10, 11, 000, 001, \dots\} \\ A &= \{0, 00, 01, 000, 001, \dots\} \\ \chi_A &= 0 \ 1 \ 0 \ 1 \ 1 \ 0 \ 0 \ 1 \ 1 \ \dots \end{aligned}$$

The function $f : \mathcal{L} \rightarrow B$, where $f(A) = \chi_A$, is a one-to-one and onto mapping (a bijection). Therefore, the sets $\mathcal{L}$ and B have the same cardinality. Since B is uncountable, $\mathcal{L}$ must be uncountable as well.

We have shown that the set of all Turing machines is countable, while the set of all languages is uncountable. This means there are vastly **more languages than there are Turing machines**. Thus, there cannot be a correspondence between the set of all languages and the set of all Turing machines, and we can conclude that some languages are not recognized by any Turing machine. $\square$

Theorem 5. *A language L is decidable if and only if both L and its complement, $\bar{L}$, are Turing-recognizable.*

Proof. Forward direction: If L is decidable, its **complement $\bar{L}$ is also decidable**. Any **decidable language is Turing-recognizable**, so we can easily see that both L and its complement $\bar{L}$ are Turing-recognizable.

Backward direction: If both L and $\bar{L}$ are Turing-recognizable, we let M_1 be the recognizer for L and M_2 be the recognizer for $\bar{L}$. **Construct** the following Turing machine M is a decider for L .

$M =$ On input w :

1. Run both M_1 and M_2 on input w in parallel.
2. If M_1 accepts, accept.
3. If M_2 accepts, reject.

Running the two machines in parallel means that M **has two tapes**, one for simulating M_1 and the other for simulating M_2 . In this case, M **takes turns** simulating one step of each machine, which continues until one of them accepts.

Now we show that M **decides L** . Every string w is either in L or $\bar{L}$. Therefore, either M_1 or M_2 must accept w . Because M halts whenever M_1 or M_2 accepts, M always halts and so it is a decider. Furthermore, it accepts all strings in L and rejects all strings not in L . So M is a decider for L , and thus L is decidable. $\square$

4.1 A non Turing-recognizable language: $\overline{A_{TM}}$

Given the above theorem, we can find a **concrete example** of a non Turing-recognizable language.

We know A_{TM} is Turing-recognizable but **undecidable**. Consider the complement of A_{TM} , $\overline{A_{TM}}$. If $\overline{A_{TM}}$ were Turing-recognizable, then since A_{TM} is also Turing-recognizable, the **theorem would imply that A_{TM} must be decidable**. This contradicts our established result. Therefore, our assumption that $\overline{A_{TM}}$ is recognizable must be false.

This shows that $\overline{A_{TM}}$ is a concrete example of a language that is not Turing-recognizable.

References

- [1] P. Linz, *An Introduction to Formal Languages and Automata*, 6th Edition, Jones & Bartlett Learning, 2016.
- [2] M. Sipser, *Introduction to the Theory of Computation*, 3rd ed. Boston, MA: Cengage Learning, 2012.